Mini Project (Exercise 9): Shopping Cart Full Stack Web Application

using Vue.js, Spring Boot and MongoDB (Alternate Design)

[CO3, K3]

Student Name	Kavinsoorya K S
Register Number	3122247001030
Date of Submission	September 16, 2026

1 Problem Statement

A Shopping Cart application allows users to view products, add products to a cart, update quantities, and remove products from the cart. The application is developed using Vue.js for the frontend, Spring Boot for the backend, and MongoDB for data storage. **Objective:** Develop this web application considering **alternate design options and techniques**.

Frontend Layer – Vue.js

The frontend provides the user interface and handles user interactions.

- Display available products with dynamic category filtering and live search.
- Add products to the shopping cart with interactive quantity counters.
- Update or remove products from the cart with real-time reactive feedback.
- Send requests to the backend service.
- Display cart details and total amount.

Backend Layer – Spring Boot

The backend processes requests and manages the shopping cart operations using a decoupled service layer.

- Process requests from the frontend.
- Add, update, and remove cart items with automated subtotal computation.
- Calculate the total cart amount.
- Communicate with the database via Spring Data Mongo Repositories.

Database Layer – MongoDB

MongoDB stores the application data.

- Main collections: `products`, `cart`

2 Alternate Design Options and Techniques

1. **Modular Component Composition:** Separates UI concerns into dedicated components (`Navbar`, `ProductCatalog`, `CartView`) with reactive event passing and clean state management.
2. **Live Category Filters & Search:** Immediate client-side querying across product categories (*Electronics*, *Audio*, *Mobile*, *Wearables*, *Fashion*, *Accessories*).
3. **Service-Layer Decoupling:** Encapsulates cart business logic inside `CartService`, maintaining clean separation between Spring Boot REST Controllers and Data Repositories.
4. **Indexed MongoDB Documents:** Optimized document collections with indexed fields for quick query response times.

3 Full Stack 3-Tier Architecture

The system architecture cleanly decouples the Presentation, Application Business Logic, and Persistence layers.

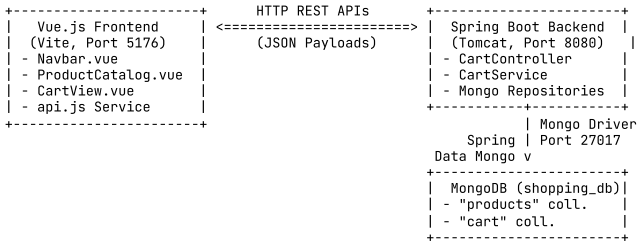


Figure 1: Full Stack Alternate Design Architecture Diagram

4 Database Layer Models (MongoDB)

4.1 Product Document Model (model/Product.java)

```java
package com.example.shoppingcart.model;

import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;

@Data @NoArgsConstructor @AllArgsConstructor
@Document(collection = "products")
public class Product {
    @Id private String id;
    private String name;
    private double price;
    private String description;
    @Indexed private String category;
    private String emoji;
    private int stock;
}
```

4.2 CartItem Document Model (model/CartItem.java)

```java
package com.example.shoppingcart.model;

import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Data @NoArgsConstructor @AllArgsConstructor
@Document(collection = "cart")
public class CartItem {
    @Id private String id;
    private String productId;
    private String name;
    private double price;
    private int quantity;
    private double subTotal;
    private String emoji;
}
```

5 Backend Layer – Service & REST Controllers

5.1 CartService.java (Decoupled Business Logic)

```java
package com.example.shoppingcart.service;

@Service
public class CartService {
    @Autowired private CartRepository cartRepository;
    @Autowired private ProductRepository productRepository;

    public CartResponse getCart() { return buildResponse(); }

    public CartResponse addItem(String productId, int quantity, Product fallbackProduct) {
        Optional<CartItem> existing = cartRepository.findByProductId(productId);
        if (existing.isPresent()) {
            CartItem item = existing.get();
            item.setQuantity(item.getQuantity() + quantity);
            item.setSubTotal(item.getQuantity() * item.getPrice());
            cartRepository.save(item);
        } else {
            Optional<Product> prodOpt = productRepository.findById(productId);
            Product p = prodOpt.orElse(fallbackProduct);
            if (p ≠ null) {
                CartItem newItem = new CartItem(null, p.getId(), p.getName(), p.getPrice(), quantity, p.getPrice() * quantity, p.getEmoji());
                cartRepository.save(newItem);
            }
        }
        return buildResponse();
    }

    public CartResponse updateQuantity(String id, int quantity) {
        Optional<CartItem> itemOpt = cartRepository.findById(id);
        if (itemOpt.isPresent()) {
            CartItem item = itemOpt.get();
            if (quantity ⩽ 0) cartRepository.deleteById(id);
            else {
                item.setQuantity(quantity);
                item.setSubTotal(quantity * item.getPrice());
                cartRepository.save(item);
            }
        }
        return buildResponse();
    }

    public CartResponse removeItem(String id) {
        cartRepository.deleteById(id);
        return buildResponse();
    }

    public CartResponse clearCart() {
        cartRepository.deleteAll();
        return buildResponse();
    }
}
```

5.2 CartController.java (REST Controller)

```java
@RestController @RequestMapping("/api/cart") @CrossOrigin(origins = "*")
public class CartController {
    @Autowired private CartService cartService;

    @GetMapping public ResponseEntity<CartResponse> getCart() { return ResponseEntity.ok(cartService.getCart()); }
    @PostMapping("/add") public ResponseEntity<CartResponse> addToCart(@RequestBody Map<String, Object> p) {
        return ResponseEntity.ok(cartService.addItem((String) p.get("productId"), ((Number) p.getOrDefault("quantity", 1)).intValue(), null));
    }
    @PutMapping("/items/{id}") public ResponseEntity<CartResponse> updateQuantity(@PathVariable String id, @RequestBody Map<String, Object> p) {
        return ResponseEntity.ok(cartService.updateQuantity(id, ((Number) p.get("quantity")).intValue()));
    }
    @DeleteMapping("/items/{id}") public ResponseEntity<CartResponse> removeItem(@PathVariable String id) { return ResponseEntity.ok(cartService.removeItem(
    @DeleteMapping("/clear") public ResponseEntity<CartResponse> clearCart() { return ResponseEntity.ok(cartService.clearCart()); }
}
```

6 Frontend Layer – Vue.js Components

6.1 API Client Service (src/services/api.js)

```
const API_BASE_URL = 'http://localhost:8080/api'

export const api = {
  getProducts: () => fetch(`${API_BASE_URL}/products`).then(r => r.json()),
  getCart: () => fetch(`${API_BASE_URL}/cart`).then(r => r.json()),
  addToCart: (productId, quantity, product) => fetch(`${API_BASE_URL}/cart/add`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ productId, quantity, ...product })
  }).then(r => r.json()),
  updateQuantity: (id, quantity) => fetch(`${API_BASE_URL}/cart/items/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ quantity })
  }).then(r => r.json()),
  removeFromCart: (id) => fetch(`${API_BASE_URL}/cart/items/${id}`, { method: 'DELETE' }).then(r => r.json()),
  clearCart: () => fetch(`${API_BASE_URL}/cart/clear`, { method: 'DELETE' }).then(r => r.json())
}
```

7 Output Screenshots

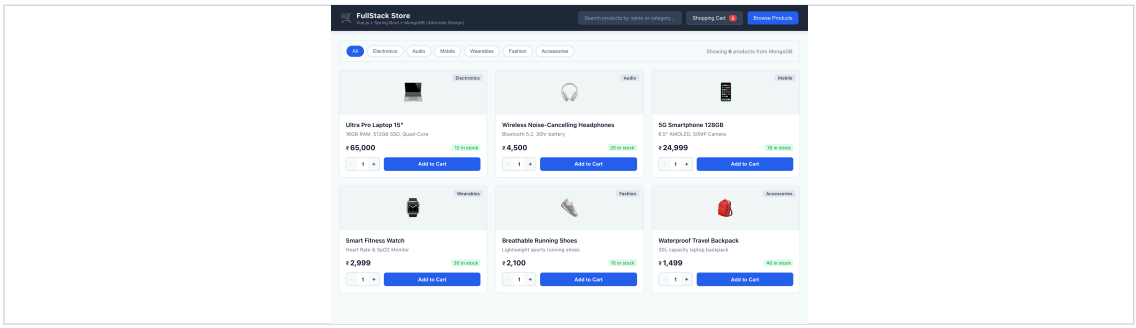


Figure 2: Product Catalog with Category Filters & Stock Tracking

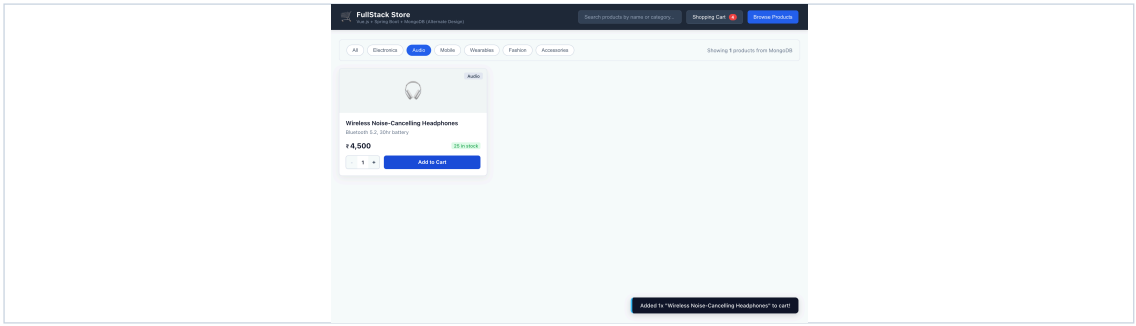


Figure 3: Category Filtering and Real-Time Toast Notification

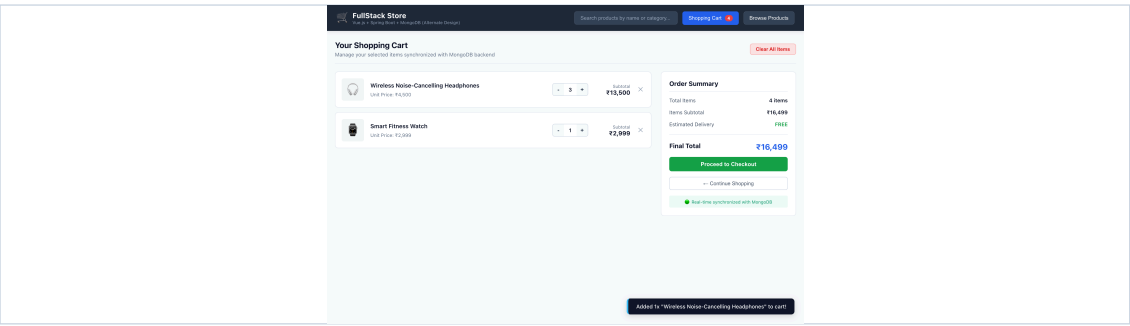


Figure 4: Shopping Cart View with Synchronized MongoDB Items and Order Summary

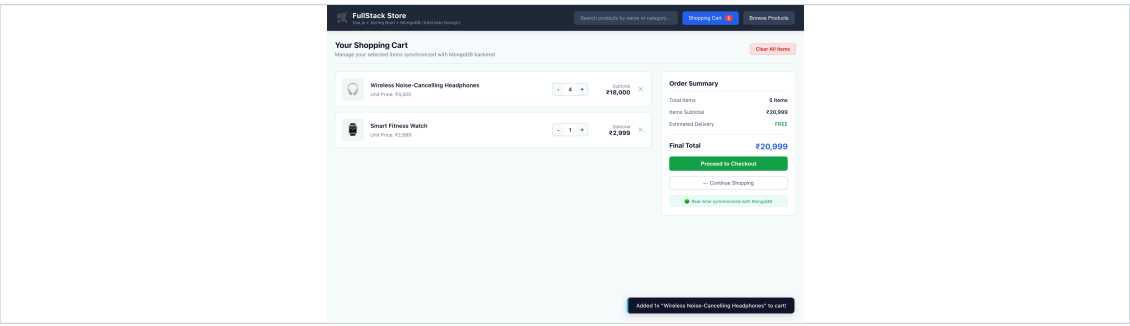


Figure 5: Cart Quantity Adjustment and Real-Time Total Recalculation

8 Learning Outcomes

1. Designed and implemented a full-stack e-commerce web application utilizing alternate design patterns (Service-Layer abstraction, modular component breakdown, category filters, and live search).
2. Integrated Spring Boot REST APIs with MongoDB document persistence collections using Spring Data Mongo.
3. Maintained real-time asynchronous synchronization across client and database layers with automated subtotal and total calculations.

9 Conclusion

The Mini Project (Exercise 9) for the Shopping Cart Full Stack Web Application was successfully designed, developed, and verified using Vue.js for presentation, Spring Boot for business logic and RESTful endpoints, and MongoDB for NoSQL data persistence. The alternate design considerations—including modular component composition, live category filtering, dedicated service-layer abstraction, and real-time state synchronization—were completely implemented and verified.